ITE 0031 | DATA STRUCTURES AND ALGORITHIMS

CAFETERIA ORDERING SYSTEM

Mini-Project Documentation

INSTRUCTOR

FARAH DIVA ALVARADO

Developer

Canales, Mark Levi Verdida

Documentation

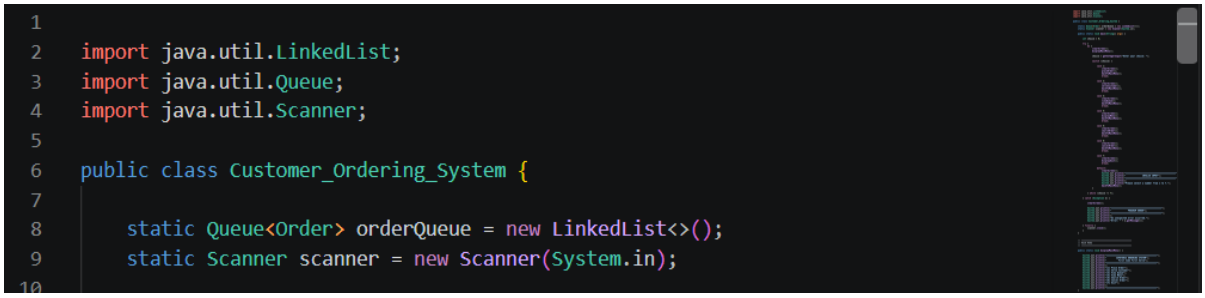
Canales, Mark Levi Verdida

BSIT 2 – B1

I. PROGRAM DESCRIPTION

Cafeteria Ordering System is a java console-based application that organize food orders in a cafeteria. It addresses the difficulty of manually tracking the order in which customers should be served. Using a First In, First Out (FIFO) queue, the application serves customers in the same order in which their orders were placed.

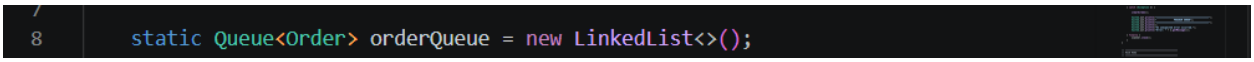
The system allows users to place an order, serve the next customer, view the waiting queue, display available food items and prices, search for a customer, cancel a selected waiting order, and exit with developer credits. The menu uses numbered choices enclosed in square brackets and provides messages for invalid input. The application stores orders in memory while it is running. It does not use a database or preserve orders after the program closes.



```
1
2   import java.util.LinkedList;
3   import java.util.Queue;
4   import java.util.Scanner;
5
6   public class Customer_Ordering_System {
7
8       static Queue<Order> orderQueue = new LinkedList<>();
9       static Scanner scanner = new Scanner(System.in);
10
```

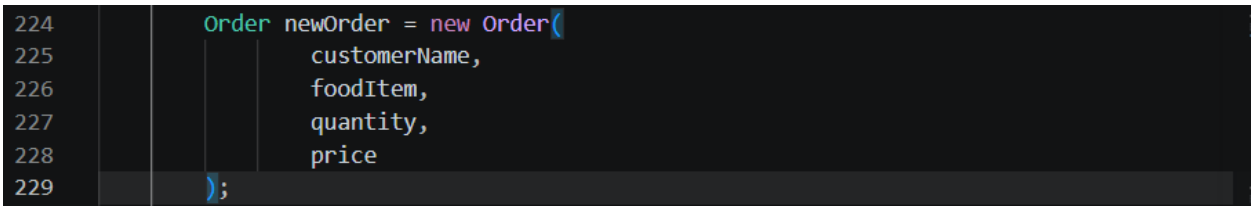
II. EXPLANATION OF DATA STRUCTURES AND ALGORITHIMS

Queue and Linked List. The application declares a Queue of Order objects and creates it using a LinkedList. A queue is suitable because the first order added must be the first one served. Linked List provides the underlying implementation for this queue.



```
8       static Queue<Order> orderQueue = new LinkedList<>();
```

FIFO and insertion. The offer **(newOder)** operation adds a new Order object to the rear of the queue. The order includes the customer's name, food item, quantity, and unit price.



```
224         Order newOrder = new Order(
225                 customerName,
226                 foodItem,
227                 quantity,
228                 price
229         );
```

FIFO and removal. The **(poll())** operation removes and returns the order at the front. The program first checks (**isEmpty()**) to avoid attempting to serve a non-existent order.

```
266         if (orderQueue.isEmpty()) {
267
268             System.out.println("There are currently no customers in the queue.");
269             return;
270         }
271
272         // FIFO: remove the first order in the queue.
273         Order order = orderQueue.poll();
```

Linear Search. The **(searchOrder())** method examines orders one at a time and compares customer names using **(equalsIgnoreCase()).** It stops at the first matching order and displays its queue position. The worst case search time is **(O(n)),** where n is the number of waiting orders.

```
357         for (Order order : orderQueue) {
358
359             if (order.getCustomerName().equalsIgnoreCase(searchName)) {
360
361                 clearScreen();
362
363                 System.out.println("==========================================");
364                 System.out.println("               ORDER FOUND");
365                 System.out.println("==========================================");
366                 System.out.println();
367
368                 System.out.println("Customer : " + order.getCustomerName());
369                 System.out.println("Food     : " + order.getFoodItem());
370                 System.out.println("Quantity : " + order.getQuantity());
371                 System.out.printf("Total    : P%.2f%n", order.getTotal());
372                 System.out.println("Position : " + position);
373
374                 found = true;
375                 break;
376             }
377
378             position++;
379         }
```

Cancel Order. The **(cancelOrder())** method scans the queue for matching names, allows the user to choose one matching order, and requests Y/N confirmation. If confirmed, **remove(selectedOrder)** deletes only that order; the remaining orders retain their relative order. The matching-order list is another LinkedList.

```
421         // Find all orders matching the customer name.
422         LinkedList<Order> matches = new LinkedList<>();
```

```
535         // Remove only the selected order.
536         // Other customers keep their original queue order.
537         orderQueue.remove(selectedOrder);
538
```

Order class. Each Order object stores **(customerName, foodItem, quantity, and price).** Getter methods provide access to these values, while **(getTotal())** calculates price multiplied by quantity.

```
676     public double getTotal() {
677         return price * quantity;
678     }
```

III. Functionality Descriptions and Screenshots

1. View Menu

Displays the five available food items and their prices.

```
120
121     public static void displayMenu() {
122
123         System.out.println("========================================");
124         System.out.println("             CAFETERIA MENU");
125         System.out.println("========================================");
126         System.out.println();
127         System.out.println("[1] Burger          - P80.00");
128         System.out.println("[2] Spaghetti       - P100.00");
129         System.out.println("[3] Fried Chicken   - P120.00");
130         System.out.println("[4] Sandwich        - P70.00");
131         System.out.println("[5] Soft Drink      - P40.00");
132         System.out.println();
133         System.out.println("========================================");
134     }
```

2. Place Order

Validates the customer name, food selection, and quantity; creates an Order; adds it to the rear; and displays a receipt and queue position.

```
141     public static void placeOrder() {
142
143         System.out.println("========================================");
144         System.out.println("               PLACE ORDER");
145         System.out.println("========================================");
146         System.out.println();
147
148         System.out.print("Enter customer name: ");
149         String customerName = scanner.nextLine().trim();
150
151         while (customerName.isEmpty()) {
152
153             System.out.println();
154             System.out.println("Customer name cannot be empty.");
155             System.out.print("Enter customer name: ");
156
157             customerName = scanner.nextLine().trim();
158         }
```

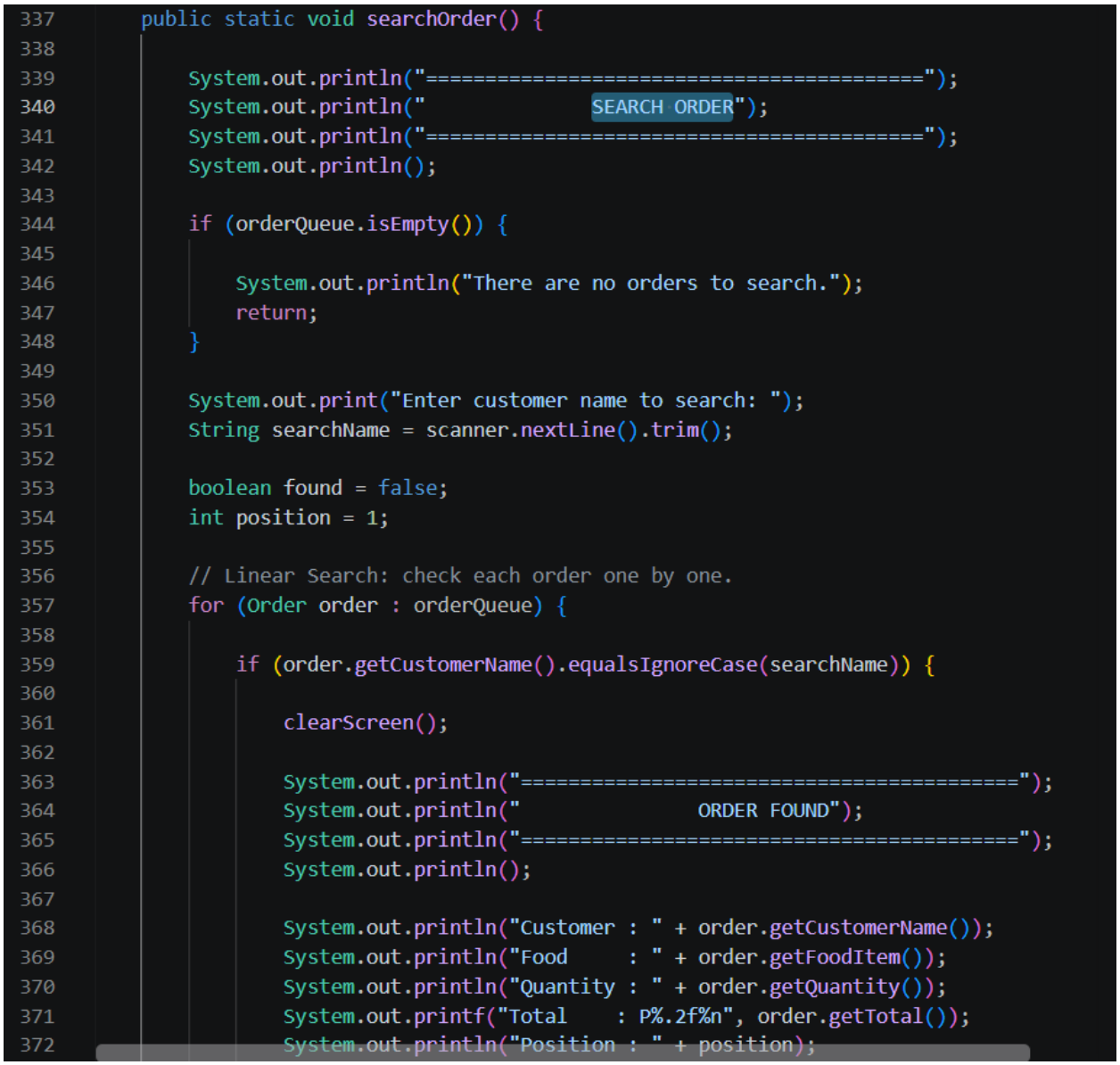
3. VIEW QUEUE

Validates the customer name, food selection, and quantity; creates an Order; adds it to the rear; and displays a receipt and queue position.

```
300     public static void viewQueue() {
301
302         System.out.println("========================================");
303         System.out.println("             CURRENT QUEUE");
304         System.out.println("========================================");
305         System.out.println();
306
307         if (orderQueue.isEmpty()) {
308
309             System.out.println("There are currently no customers in the queue.");
310             return;
311         }
312
313         int position = 1;
314
315         for (Order order : orderQueue) {
316
317             System.out.println("Position #" + position);
318             System.out.println("Customer : " + order.getCustomerName());
319             System.out.println("Food     : " + order.getFoodItem());
320             System.out.println("Quantity : " + order.getQuantity());
321             System.out.printf("Total    : P%.2f%n", order.getTotal());
322
323             System.out.println("----------------------------------------");
324
325             position++;
326         }
327
328         System.out.println();
329         System.out.println("Total Customers Waiting: " + orderQueue.size());
330     }
331
```

4. SEARCH ORDER

Finds the first matching customer name and displays order details and position.



```
337     public static void searchOrder() {
338
339         System.out.println("========================================");
340         System.out.println("              SEARCH ORDER");
341         System.out.println("========================================");
342         System.out.println();
343
344         if (orderQueue.isEmpty()) {
345
346             System.out.println("There are no orders to search.");
347             return;
348         }
349
350         System.out.print("Enter customer name to search: ");
351         String searchName = scanner.nextLine().trim();
352
353         boolean found = false;
354         int position = 1;
355
356         // Linear Search: check each order one by one.
357         for (Order order : orderQueue) {
358
359             if (order.getCustomerName().equalsIgnoreCase(searchName)) {
360
361                 clearScreen();
362
363                 System.out.println("========================================");
364                 System.out.println("               ORDER FOUND");
365                 System.out.println("========================================");
366                 System.out.println();
367
368                 System.out.println("Customer : " + order.getCustomerName());
369                 System.out.println("Food     : " + order.getFoodItem());
370                 System.out.println("Quantity : " + order.getQuantity());
371                 System.out.printf("Total    : P%.2f%n", order.getTotal());
372                 System.out.println("Position : " + position);
```



```
373
374                 found = true;
375                 break;
376             }
377
378             position++;
379         }
380
381         if (!found) {
382
383             clearScreen();
384
385             System.out.println("========================================");
386             System.out.println("             ORDER NOT FOUND");
387             System.out.println("========================================");
388             System.out.println();
389
390             System.out.println("No order was found for: " + searchName);
391         }
392     }
393
```

5. SERVE CUSTOMER

Removes and displays the first waiting order using poll().

```java
public static void serveCustomer() {

    System.out.println("========================================");
    System.out.println("            SERVE CUSTOMER");
    System.out.println("========================================");
    System.out.println();

    if (orderQueue.isEmpty()) {

        System.out.println("There are currently no customers in the queue.");
        return;
    }

    // FIFO: remove the first order in the queue.
    Order order = orderQueue.poll();

    System.out.println("Now Serving");
    System.out.println();
    System.out.println("----------------------------------------");

    System.out.println("Customer : " + order.getCustomerName());
    System.out.println("Food     : " + order.getFoodItem());
    System.out.println("Quantity : " + order.getQuantity());
    System.out.printf("Total    : P%.2f%n", order.getTotal());

    System.out.println("----------------------------------------");
    System.out.println();
    System.out.println("Order served successfully!");

    if (orderQueue.isEmpty()) {
        System.out.println("No more customers are waiting.");
    } else {
        System.out.println("Customers remaining: " + orderQueue.size());
    }
```

6. CANCEL ORDER

Shows matching orders, accepts a selection, asks for confirmation, and removes the selected order if the answer is Y.

```
528         System.out.println("========================================");
529         System.out.println("              CANCEL ORDER");
530         System.out.println("========================================");
531         System.out.println();
532 
533         if (confirmation.equalsIgnoreCase("Y")) {
534 
535             // Remove only the selected order.
536             // Other customers keep their original queue order.
537             orderQueue.remove(selectedOrder);
538 
539             System.out.println("Order cancelled successfully!");
540             System.out.println();
541             System.out.println("Customer : " + selectedOrder.getCustomerName());
542             System.out.println("Food     : " + selectedOrder.getFoodItem());
543             System.out.println();
544             System.out.println("Remaining orders: " + orderQueue.size());
545 
546         } else {
547 
548             System.out.println("Cancellation aborted.");
549             System.out.println("The order remains in the queue.");
550         }
551     }
552 
```

7. CANCEL CONFIRMATION

Displays the selected order and asks whether to cancel it using Y/N.

```
498         System.out.println("=========================================");
499         System.out.println("           CONFIRM CANCELLATION");
500         System.out.println("=========================================");
501         System.out.println();
502 
503         System.out.println("Customer : " + selectedOrder.getCustomerName());
504         System.out.println("Food     : " + selectedOrder.getFoodItem());
505         System.out.println("Quantity : " + selectedOrder.getQuantity());
506         System.out.printf("Total    : P%.2f%n", selectedOrder.getTotal());
507 
508         System.out.println();
509 
510         String confirmation;
511 
512         do {
513 
514             System.out.print("Are you sure you want to cancel? (Y/N): ");
515             confirmation = scanner.nextLine().trim();
516 
517             if (!confirmation.equalsIgnoreCase("Y")
518                     && !confirmation.equalsIgnoreCase("N")) {
519 
520                 System.out.println("Invalid input. Please enter Y or N.");
521             }
522 
523         } while (!confirmation.equalsIgnoreCase("Y")
524                 && !confirmation.equalsIgnoreCase("N"));
525 
526         clearScreen();
```

8. EXIT AND DEVELOPER CREDITS

Ends the program and displays the sole developer, degree program, and section.

```
613     public static void displayExit() {
614
615         System.out.println("========================================");
616         System.out.println("        CAFETERIA ORDERING SYSTEM");
617         System.out.println("========================================");
618         System.out.println();
619         System.out.println("Thank you for using the program!");
620         System.out.println();
621         System.out.println("========================================");
622         System.out.println("            DEVELOPER CREDITS");
623         System.out.println("========================================");
624         System.out.println();
625         System.out.println("Developer      : Mark Levi V. Canales");
626         System.out.println("Role           : Developer");
627         System.out.println("Degree Program : BS Information Technology");
628         System.out.println("Section        : BSIT 2 - B1");
629         System.out.println();
630         System.out.println("========================================");
631         System.out.println("              PROGRAM ENDED");
632         System.out.println("========================================");
633     }
634 }
```

IV. EXCEPTION HANDLING AND INPUT VALIDATION

The **(getIntegerInput())** method uses (**Integer.parseInt())** inside a try-catch block. When a user enters text that cannot be converted into an integer, Java raises **(NumberFormatException)**. The catch block displays a helpful error message, and the loop asks for input again, allowing the program to continue.

```
562             try {
563
564                 System.out.print(prompt);
565
566                 String input = scanner.nextLine().trim();
567
568                 return Integer.parseInt(input);
569
570             } catch (NumberFormatException e) {
571
572                 System.out.println();
573                 System.out.println("Invalid input!");
574                 System.out.println("Please enter a whole number.");
575                 System.out.println();
576             }
```

```
==========================================
Enter your choice:
hello

Invalid input!
Please enter a whole number.

Enter your choice:
>
```

Other validation checks reject blank customer names, food selections outside 1-5, nonpositive quantities, invalid cancellation selections, and confirmation answers other than Y or N. A broader **(catch(Exception e))** in main reports unexpected errors, and finally closes the Scanner. The application also uses **(clearScreen())**and **(backToMainMenu())** to make navigation easier.

```
601     public static void clearScreen() {
602
603         // ANSI escape sequences for supported terminals.
604         System.out.print("\033[H\033[2J");
605         System.out.flush();
606     }
```

V. PROJECT LEARNING OUTCOMES

Developing the Cafeteria Ordering System helped me understand how data structures and algorithms solve practical problems. I learned that a Queue is appropriate for managing customer orders because its FIFO behavior supports fair, first-come-first-served processing. I learned how to add orders with offer(), serve customers with poll(), inspect a collection without removing items, and selectively cancel an order using remove(). Implementing Linear Search improved my understanding of how a program checks each element until it finds a match, and why the time needed for searching depends on the number of stored orders. The project also improved my ability to validate user input and handle errors using try-catch. I learned to separate program functions into methods, design a clearer menu, and provide confirmation before an important action such as cancellation.